

Universidad del Valle de Guatemala
Facultad de Ingeniería
Departamento de Ciencias de la Computación
CC3084 — Data Science

Laboratorio 5
Clasificación de tweets usando minería de texto

Integrantes

Hugo Barillas — carné 23556
Esteban Cárcamo — carné 23016
Ernesto Ascencio — carné 23009

Repositorio del proyecto
<https://github.com/ecarcamo/CC3084-DATA-SCIENCE/tree/lab5>

Semestre II — 2026

1. Introducción

El presente informe documenta el trabajo que realizamos sobre el conjunto de datos Natural Language Processing with Disaster Tweets, publicado por Kaggle. El problema consiste en determinar, a partir del texto de un tweet, si este se refiere a un desastre real o si emplea vocabulario de catástrofe en algún otro sentido. La distinción no es trivial: expresiones como “this song is fire” o “my car got wrecked” comparten vocabulario con reportes de incendios y accidentes reales, y esa ambigüedad atraviesa todo el análisis que presentamos.

Organizamos el trabajo en ocho cuadernos de Jupyter que se ejecutan en secuencia, apoyados en módulos de Python reutilizables que concentran la lógica. Cada cuaderno corresponde a uno de los ejercicios del enunciado y produce los insumos del siguiente. Todo el código, junto con las instrucciones de reproducción, está disponible en el repositorio enlazado en la portada.

Adoptamos un criterio metodológico que conviene declarar desde el inicio, porque explica varias de las decisiones que tomamos más adelante. En lugar de confiar en una sola partición de los datos para comparar modelos, repetimos las comparaciones sobre veinte particiones aleatorias distintas y aplicamos pruebas estadísticas pareadas. Esa precaución nos salvó al menos una vez de elegir el modelo equivocado, como se detalla en la sección 6.

2. Descripción del conjunto de datos

El archivo train.csv contiene 7,613 tweets con cinco columnas. La columna id identifica cada tweet, keyword guarda una palabra clave asociada, location indica la ubicación desde la que se envió, text contiene el texto completo y target es la etiqueta que vale 1 si el tweet describe un desastre real y 0 si no. El enunciado menciona más de 10,500 filas, cifra que corresponde a la suma de train.csv y test.csv; como este último no incluye la columna target, no permite evaluar nada y trabajamos únicamente con el primero.

Al revisar la completitud de los datos encontramos que location tiene 33.27% de valores ausentes, lo que era esperable porque es un campo opcional de Twitter que además admite texto libre. La columna keyword apenas tiene 0.80% de ausencias, y ni text ni target presentan valores nulos. La tabla 1 resume esta situación.

Columna	Tipo	Nulos	Observación
id	entero	0.00%	identificador
keyword	texto	0.80%	palabra clave
location	texto	33.27%	texto libre
text	texto	0.00%	contenido del tweet
target	entero	0.00%	etiqueta a predecir

Tabla 1. Estructura y completitud del conjunto de datos.

En cuanto al balance entre clases, que la figura 1 ilustra, 57% de los tweets corresponden a la categoría de no desastre y 43% a desastre real. Es un desbalance moderado que no exige técnicas agresivas de remuestreo, aunque sí influyó en la métrica que elegimos para comparar modelos y en el uso de ponderación por clase durante el entrenamiento.

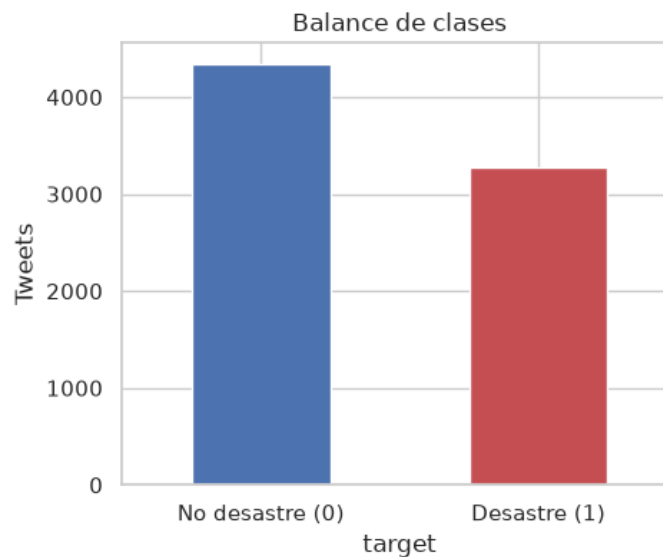


Figura 1. Distribución de las clases en el conjunto de entrenamiento.

3. Limpieza y preprocesamiento

Construimos un pipeline de limpieza de siete pasos que se aplican en orden estricto: conversión a minúsculas, eliminación de URLs mediante expresiones regulares, eliminación de los caracteres especiales arroba, numeral y apóstrofe, eliminación de caracteres fuera del rango ASCII, eliminación de signos de puntuación, eliminación de números y, finalmente, eliminación de palabras vacías en inglés. Guardamos el resultado de cada paso como un archivo CSV independiente, de modo que se puede inspeccionar el efecto de cada transformación por separado.

Los módulos que utilizamos son `re` y `string` de la biblioteca estándar de Python, `nltk` para la lista de palabras vacías y `pandas` para la manipulación de los datos. El pipeline completo reside en el módulo `src/limpieza.py` y se invoca desde el cuaderno de preprocesamiento.

El efecto agregado de la limpieza se aprecia en la figura 2. El largo promedio del tweet pasa de 101 a 63.8 caracteres, una reducción del 37%. Los dos pasos que más recortan son la eliminación de palabras vacías, que quita el 22% del texto que quedaba en ese momento, y la eliminación de URLs, porque los enlaces acortados de Twitter ocupan alrededor de 14 caracteres cada uno.

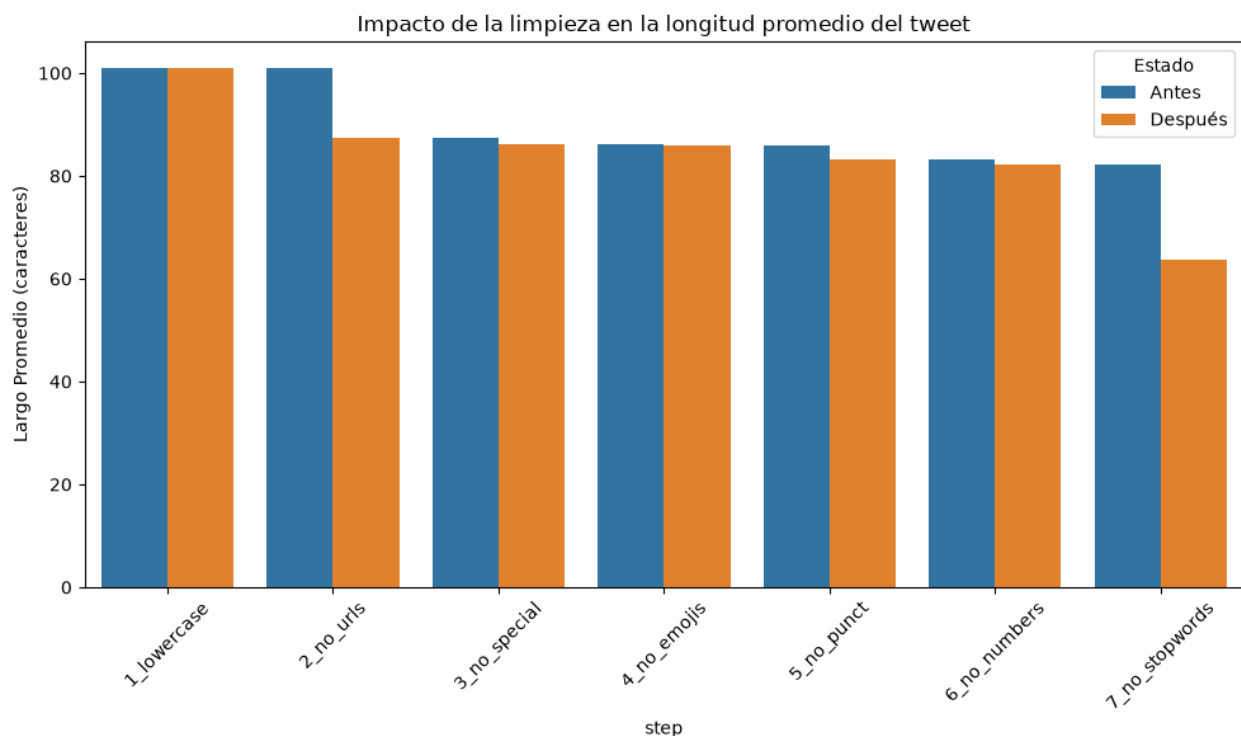


Figura 2. Largo promedio del tweet antes y después de cada paso de limpieza.

Dos decisiones de esta etapa merecen comentario. La primera se refiere al número 911, que el enunciado señala explícitamente como caso a valorar. Al medirlo encontramos que aparece en apenas 10 de los 7,613 tweets, aunque 8 de ellos corresponden a desastres reales. Es una señal fuerte pero de volumen despreciable, así que optamos por eliminar todos los números sin añadir una excepción que complicaría el pipeline a cambio de muy poco.

La segunda es que quitar los apóstrofes antes de eliminar las palabras vacías convierte `don't` en `dont` e `I'm` en `im`, formas que ya no coinciden con la lista de `nltk` y por lo tanto sobreviven a la limpieza. Las encontramos después entre las palabras más frecuentes de la clase no desastre, donde terminan funcionando como marcador de lenguaje coloquial. Decidimos conservar ese comportamiento porque resultó informativo, pero lo dejamos anotado por transparencia.

4. Análisis de frecuencias y n-gramas

Extrajimos los n-gramas más frecuentes de cada clase sin partir de una lista de palabras predefinida, para dejar que los datos indicaran qué términos separan mejor las categorías. Utilizamos CountVectorizer de scikit-learn, que permite fijar el rango de n-gramas y devuelve la matriz de conteos sobre la que sumamos por término.

La figura 3 muestra el contraste entre ambas clases. En la clase de no desastre predominan términos coloquiales de uso general como like, im, amp, new, get y dont, que aparecen con frecuencia alta en cualquier conversación y por eso aportan poca capacidad de separación. La clase de desastre concentra en cambio vocabulario específico de eventos: fire, news, disaster, california, suicide, police y killed.

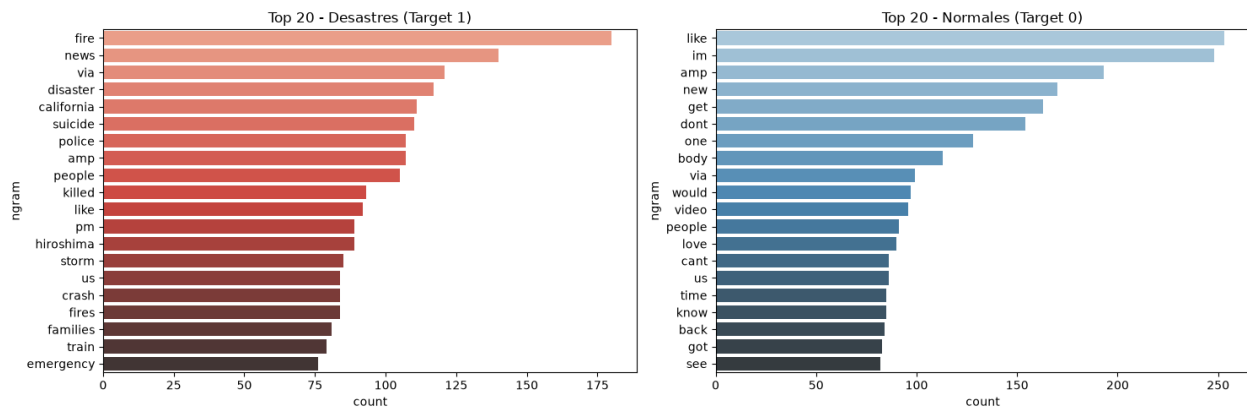


Figura 3. Veinte unigramas más frecuentes en cada categoría.

Respecto a la pregunta del enunciado sobre si vale la pena explorar bigramas o trigramas, la figura 4 nos permitió formar una primera opinión. Los bigramas agrupan conceptos legibles como suicide bomber, que aparece 59 veces, y northern california, con 41 apariciones. A nivel descriptivo muestran un contexto que el unigrama suelto no deja ver.

Los trigramas se comportan de otra manera. Secuencias como latest homes razed se repiten unas 30 veces de forma casi idéntica, algo que no corresponde a lenguaje espontáneo sino a cuentas automatizadas y cadenas de noticias replicando el mismo titular. Un modelo entrenado sobre ellos se ajustaría a las noticias puntuales del periodo en que se recolectó el conjunto de datos.

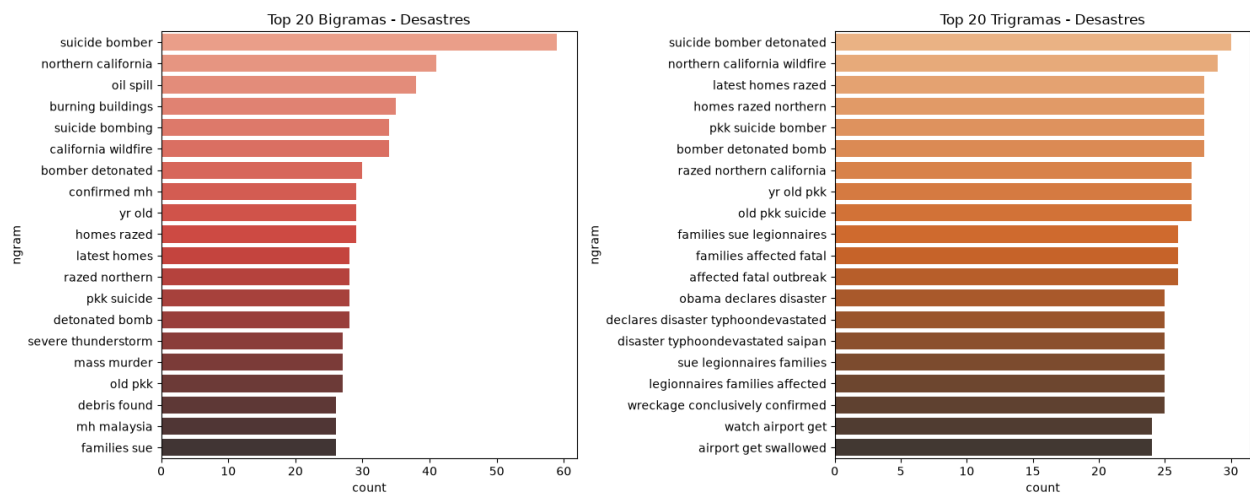


Figura 4. Bigramas y trigramas más frecuentes en la clase de desastre real.

Dejamos deliberadamente abierta la pregunta de si esta lectura descriptiva se traduce en mejor desempeño predictivo, porque que un n-grama resulte legible para nosotros no garantiza que ayude al modelo. La comprobación se hizo en la sección 6 y el resultado nos sorprendió.

5. Análisis exploratorio

5.1 Cruce entre la palabra clave y la etiqueta

La columna keyword resultó ser la variable estructurada con más señal. Al calcular la proporción de desastres reales asociada a cada palabra clave, restringiéndonos a las que tienen al menos diez observaciones para evitar conclusiones sobre casos aislados, encontramos un contraste muy marcado que resume la tabla 2.

Más asociadas a desastre	Tasa	Menos asociadas	Tasa
debris	1.000	aftershock	0.000
derailment	1.000	body bags	0.024
wreckage	1.000	ruin	0.027
outbreak	0.975	blazing	0.029
oil spill	0.974	body bag	0.030
typhoon	0.974	electrocute	0.031

Tabla 2. Palabras clave con mayor y menor proporción de desastres reales.

Palabras como debris, derailment y wreckage acompañan siempre a un desastre real, mientras que aftershock, ruin o blazing casi nunca lo hacen, porque suelen emplearse en sentido figurado

o aparecen en contextos de música y espectáculos. La palabra clave es por tanto una señal aprovechable, aunque parcial.

5.2 Ubicación y largo del texto

La columna location resultó demasiado ruidosa para usarse directamente. Conviven en ella variantes del mismo lugar, como USA, United States y NYC, junto a entradas que ni siquiera son geográficas, como Worldwide o Everywhere. Normalizarla exigiría geocodificación o un mapeo manual, así que no la incorporamos como variable del modelo.

El largo del texto sí mostró una diferencia consistente entre clases, visible en la figura 5. Los tweets de desastre real promedian 108.1 caracteres y 15.2 palabras, frente a 95.7 caracteres y 14.7 palabras de la otra categoría. La interpretación que nos parece más razonable es que un reporte informativo necesita más detalles, como el lugar y el tipo de evento, mientras que un comentario casual o metafórico se resuelve en menos palabras.

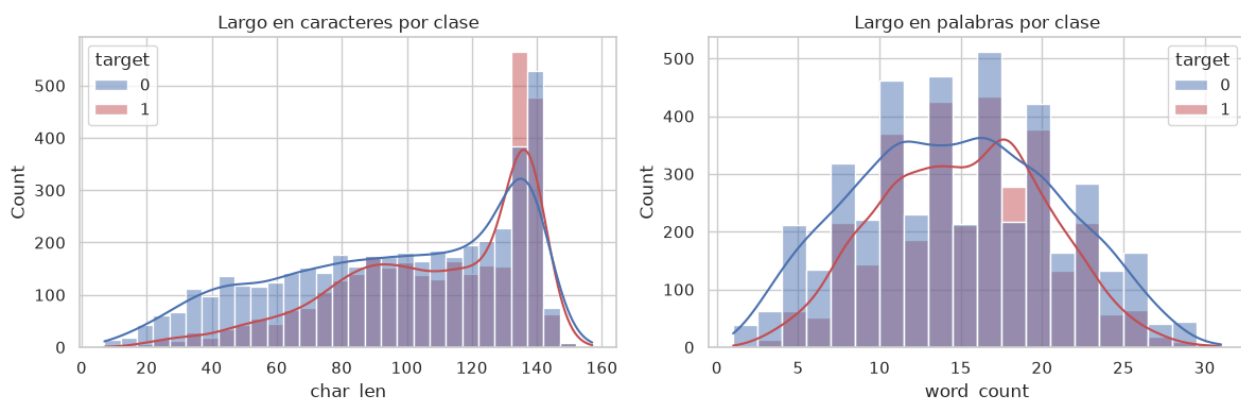


Figura 5. Distribución del largo del tweet en caracteres y en palabras, por categoría.

5.3 Nubes de palabras y vocabulario compartido

Las nubes de palabras de la figura 6 confirman visualmente el patrón que ya habían insinuado los conteos. La nube de la clase de desastre está cargada de eventos concretos como fire, flood, storm y emergency, mientras que la de no desastre muestra un vocabulario más disperso y conversacional.



Figura 6. Nubes de palabras del corpus completo y de cada categoría.

El enunciado pide discutir las palabras que tienen presencia en todas las categorías. Al comparar los cien términos más frecuentes de cada clase encontramos 26 en común, entre ellos like, new, via, get, people, time y today. Son términos genéricos de alta frecuencia que por sí solos no permiten decidir la clase, y anticipamos que un esquema de ponderación como TF-IDF les asignaría poco peso justamente por aparecer en todas partes. El poder discriminante se concentra en los 74 términos exclusivos de la clase de desastre, como earthquake, killed, flood y derailment, y en los exclusivos de la otra clase, que reflejan uso coloquial.

6. Modelos de clasificación

6.1 Diseño del experimento

Separamos los datos en 80% para entrenamiento y 20% para prueba, de forma estratificada para conservar el balance de clases en ambos lados. La partición quedó en 6,090 tweets de entrenamiento y 1,523 de prueba, con una proporción de desastres de 0.4297 y 0.4294 respectivamente.

Cada modelo se construyó como un Pipeline que encadena el vectorizador TF-IDF con el clasificador. Esta decisión no es cosmética: al encapsularlos juntos, el vectorizador se ajusta únicamente con el subconjunto de entrenamiento en cada corte de la validación cruzada. Si hubiéramos vectorizado el conjunto completo de una vez, el vocabulario y los pesos IDF del conjunto de prueba habrían pasado al entrenamiento y las métricas habrían salido infladas.

Sobre cómo abordar el contexto, que es la pregunta explícita del enunciado, trabajamos en tres frentes. El primero fueron los n-gramas, entrenando cada algoritmo con unigramas, con unigramas más bigramas y con unigramas más bigramas y trigramas. El segundo fue la ponderación TF-IDF, que castiga automáticamente los términos compartidos entre clases sin necesidad de mantener una lista negra manual. El tercero fue el parámetro `min_df`, que descarta términos aparecidos en un solo tweet y elimina así errores de escritura y ruido que el modelo memorizaría.

Probamos cuatro algoritmos. Naive Bayes multinomial como línea base clásica de clasificación de texto, con suavizado de Laplace de 1.0. Regresión logística por ser lineal e interpretable, con `C` igual a 1.0 y un máximo de 1,000 iteraciones. Máquina de vectores de soporte lineal, fuerte en espacios dispersos de alta dimensión como el que produce TF-IDF, con `C` igual a 1.0. Y bosque aleatorio con 300 árboles y un mínimo de dos muestras por hoja, como control no lineal frente a los tres anteriores.

6.2 Comparación de algoritmos y representaciones

La tabla 3 recoge el desempeño de los cuatro algoritmos con la representación de unigramas, que resultó la mejor de las tres. Reportamos las métricas sobre la clase positiva, es decir la de

desastre real.

Modelo	Exactitud	Precisión	Recall	F1	ROC-AUC
Regresión logística	0.8273	0.8497	0.7263	0.7832	0.8699
Naive Bayes	0.8188	0.8462	0.7064	0.7700	0.8665
SVM lineal	0.7965	0.7756	0.7401	0.7574	0.8511
Bosque aleatorio	0.7932	0.7792	0.7232	0.7502	0.8601

Tabla 3. Desempeño de los cuatro algoritmos con representación de unigramas.

La regresión logística gana de forma consistente en las tres representaciones que probamos. El bosque aleatorio queda por debajo de los modelos lineales, lo cual era esperable en datos TF-IDF dispersos y de alta dimensión, donde los árboles fragmentan el espacio sin aprovechar su geometría. Naive Bayes logra la precisión más alta pero el recall más bajo, es decir que es conservador y solo marca desastre cuando tiene mucha evidencia.

6.3 El contexto no ayudó como esperábamos

El resultado que más nos sorprendió fue el efecto de los n-gramas. Como muestra la figura 7, añadir bigramas casi duplica el número de características, de 5,375 a 9,431, y añadir trigramas lo lleva a 12,208, pero el mejor F1 de prueba no mejora: baja de 0.783 con unigramas a 0.778 con bigramas y a 0.772 con trigramas.

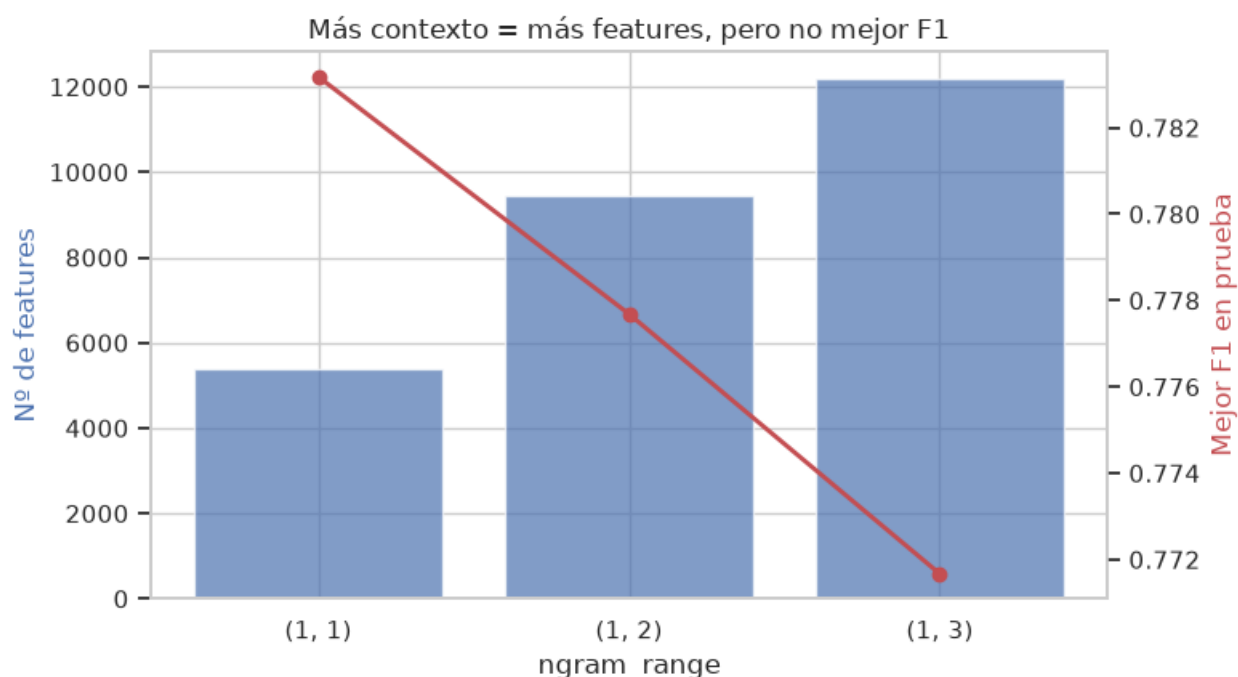


Figura 7. Número de características y mejor F1 obtenido según el rango de n-gramas.

Encontramos tres explicaciones que nos parecen complementarias. La primera es que los tweets son muy cortos: después de la limpieza quedan alrededor de siete palabras útiles, de modo que hay poco espacio para que una secuencia de dos o tres palabras aporte información que las palabras sueltas no den ya. La segunda es que los bigramas son dispersos por construcción, y un bigrama informativo pero raro, como *suicide bomber* con sus 59 apariciones en 7,613 tweets, casi nunca se activa al evaluar un tweet nuevo. La tercera, y probablemente la de fondo, es que la señal que distingue las clases es de vocabulario y no de sintaxis.

Esto responde la pregunta que habíamos dejado abierta en la sección 4. Sobre los trigramas confirma nuestra sospecha de que perjudican. Sobre los bigramas la corrige: construyen conceptos legibles para un lector humano, y eso no se traduce en poder predictivo.

6.4 Afinado y una contradicción entre criterios

Sobre la regresión logística ejecutamos una búsqueda en malla de 120 combinaciones, optimizando el F1 de la clase de desastre por validación cruzada de cinco pliegues. Incluimos deliberadamente la opción de ponderación balanceada por clase, porque en este problema dejar pasar un desastre real resulta más costoso que una falsa alarma, y el balanceo empuja al modelo hacia mayor recall.

El resultado nos puso en una situación incómoda. La configuración afinada mejoraba el F1 de validación cruzada, que pasaba de 0.7393 a 0.7552, pero empeoraba el del conjunto de prueba, que caía de 0.7832 a 0.7692. Los dos criterios apuntaban a modelos distintos. Quedarnos con el que se veía mejor en la prueba habría sido un error, porque estaríamos eligiendo sobre el mismo conjunto con el que después reportamos el desempeño, y la estimación final saldría sesgada al alza.

Sospechamos que la causa era ruido de muestreo, ya que con 1,523 tweets de prueba el error estándar del F1 ronda 0.015, suficiente para invertir el orden de dos modelos que difieren en 0.014. En vez de decidir a ojo, repetimos el experimento sobre veinte particiones aleatorias distintas comparando ambos candidatos de forma pareada. La figura 8 muestra el resultado.

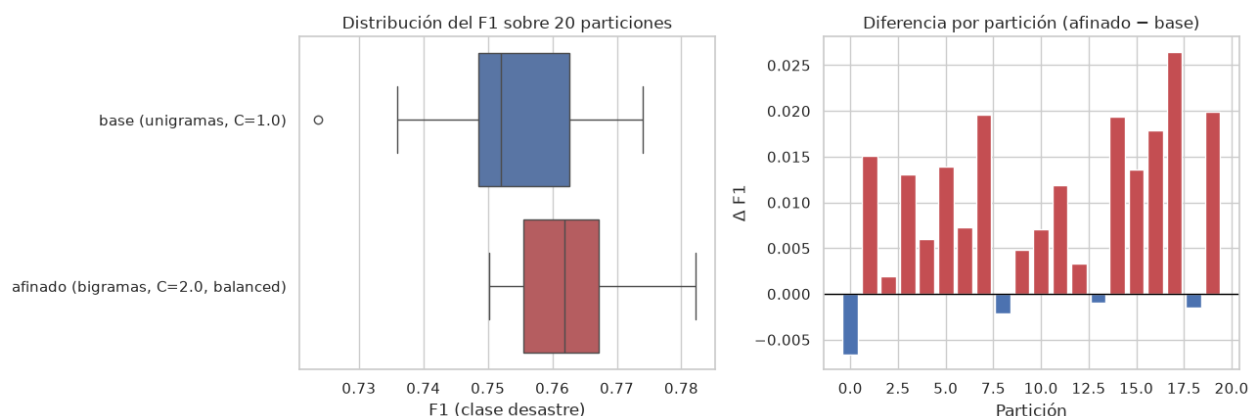


Figura 8. Distribución del F1 sobre veinte particiones y diferencia partición a partición.

El modelo afinado gana en 16 de las 20 particiones, con una diferencia media de 0.0095 en F1 y un valor p de 0.00014 en la prueba t pareada. La diferencia es pequeña pero sistemática. La partición individual que favorecía al modelo base resultó ser una de las cuatro excepciones. El modelo afinado presenta además menor desviación estándar, 0.0089 frente a 0.0115, lo que indica que es más estable ante el cambio de partición. Sin esta comprobación habríamos seleccionado el modelo equivocado.

6.5 Modelo seleccionado

El modelo elegido es una regresión logística sobre TF-IDF de unigramas y bigramas, con C igual a 2.0, ponderación balanceada por clase, min_df igual a 1 y sin escalado sublineal de frecuencias. La tabla 4 presenta su desempeño sobre el conjunto de prueba y la figura 9 su matriz de confusión.

Métrica	No desastre	Desastre real	Global
Precisión	0.821	0.782	0.802
Recall	0.841	0.757	0.799
F1	0.831	0.769	0.800
Exactitud			0.805
ROC-AUC			0.869

Tabla 4. Desempeño del modelo seleccionado sobre los 1,523 tweets de prueba.

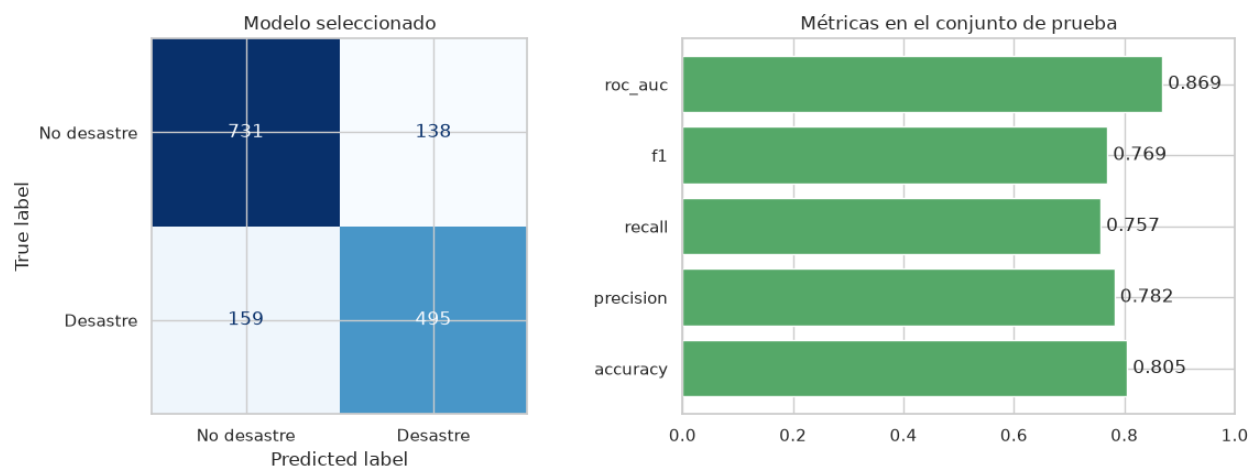


Figura 9. Matriz de confusión y métricas del modelo seleccionado.

Elegimos el F1 de la clase de desastre como criterio de selección en lugar de la exactitud. Con clases desbalanceadas, la exactitud premia acertar la clase mayoritaria, y en un problema de detección de desastres interesa más no dejar pasar los positivos reales.

Como el clasificador es lineal, sus coeficientes son directamente legibles y sirven para verificar que aprendió algo sensato. La figura 10 los muestra. Los términos que más empujan hacia desastre son hiroshima, fire, fires, california, killed, police, suicide, bombing, earthquake, wildfire y storm. Los que empujan en sentido contrario son im, love, new, body, wrecked, harm, bloody, ruin y blew. Nos pareció notable que este segundo grupo esté formado precisamente por los usos metafóricos que habíamos identificado de forma cualitativa en el análisis exploratorio: el modelo aprendió por su cuenta la distinción que nosotros habíamos descrito.

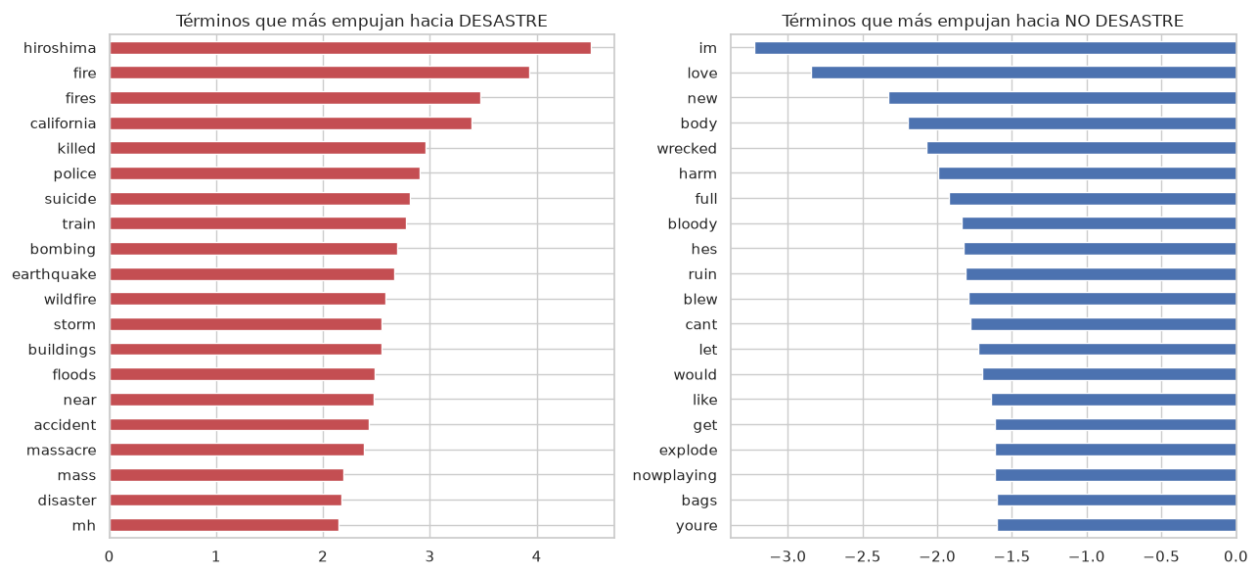


Figura 10. Términos con mayor peso hacia cada categoría en el modelo lineal.

7. Función de clasificación de tweets nuevos

Construimos una función que recibe un tweet tal como llega de Twitter, con mayúsculas, URLs, hashtags, menciones y emojis, y devuelve si describe un desastre real. La restricción de que el texto entre sin preprocesar es importante, porque obliga a que el tweet nuevo atravesase exactamente la misma transformación que atravesaron los tweets de entrenamiento. Si el texto de entrenamiento pasó por siete pasos y el tweet nuevo pasa por seis, o por los mismos siete en otro orden, el vocabulario deja de coincidir y las predicciones se degradan de forma silenciosa.

Lo resolvimos por dos vías. La limpieza se toma de la misma constante que generó el archivo de entrenamiento, de modo que no exista una segunda copia del pipeline capaz de quedar desactualizada. Y la vectorización viaja dentro del Pipeline serializado, con el vocabulario y los pesos IDF congelados durante el entrenamiento, así que el objeto guardado no es solo el clasificador sino la transformación completa.

Para verificar que el alineamiento se sostiene, pasamos los 1,523 tweets crudos del conjunto de prueba por la función completa y comparamos contra las métricas obtenidas sobre texto ya limpio. La exactitud end-to-end resultó 0.8050 y el F1 0.7692, idénticos hasta el cuarto decimal a los reportados en la sección 6. La función reproduce el desempeño del modelo partiendo de texto sin preprocesar, que era exactamente lo que había que demostrar.

La función devuelve además la probabilidad estimada, un nivel de confianza y el texto limpio que el modelo realmente vio, lo que resultó muy útil para depurar: cuando una predicción sorprende, casi siempre se entiende al observar qué quedó del tweet después de la limpieza. Los tweets que quedan vacíos tras la limpieza, como los formados solo por un enlace, se reportan

como indeterminados en lugar de forzar una etiqueta que estaría basada únicamente en el sesgo del modelo.

La tabla 5 recoge una selección de los casos de prueba con los que evaluamos la función, elegidos para cubrir los patrones que el análisis exploratorio había identificado como difíciles.

Tweet	Predicción	Prob.
Forest fire near La Ronge Sask. Canada	Desastre	0.894
Evacuation orders issued as wildfire spreads	Desastre	0.945
this new album is FIRE @spotify #banger	No desastre	0.471
I'm on fire today at the gym	Desastre	0.500
that party was fire	Desastre	0.768
My car got wrecked in the parking lot	No desastre	0.215
Beautiful sunset at the beach today	No desastre	0.283

Tabla 5. Casos de prueba de la función de clasificación.

Tres de los casos merecen atención y los tres son el mismo problema. El tweet sobre el álbum se clasifica correctamente, pero con probabilidad 0.471, apenas por debajo del umbral: la palabra fire aporta 1.34 y por sí sola bastaría para cruzar la línea, y lo que salva la predicción es que new y album arrastran en sentido contrario. El tweet del gimnasio ya falla, aunque por cuatro diezmilésimas. Y el del party falla de forma contundente, con 0.768, porque al quedar reducido a dos palabras tras la limpieza, fire con su coeficiente de 2.10 solo encuentra a party enfrente.

El patrón que forman los tres es claro: mientras más corto es el tweet metafórico, peor falla. La limpieza deja pocos términos y, si uno de ellos carga un coeficiente alto, no queda nada que lo contrapesa. Es la contracara del hallazgo sobre los tweets cortos de la sección 6, ahora visible como modo de falla concreto. La raíz del problema es que un modelo de bolsa de palabras no distingue el sentido literal del figurado, porque no modela la relación entre fire y gym, solo suma pesos independientes.

8. Análisis de sentimiento

Para determinar qué palabras son positivas, negativas o neutras utilizamos VADER, un analizador basado en léxico y reglas incluido en nltk y diseñado específicamente para texto de redes sociales. Cada término de su léxico trae una valencia entre -4 y +4 asignada por anotadores humanos. Lo preferimos sobre alternativas como TextBlob porque, además del léxico, aplica reglas que importan en tweets: negación, intensificadores, mayúsculas sostenidas,

signos de exclamación repetidos y emoticones. La referencia completa de la herramienta aparece en la sección 12.

8.1 Sobre qué texto se calcula el sentimiento

La primera decisión fue si analizar el texto crudo o el texto limpio de la sección 3. Parecía un detalle de implementación y resultó determinar el resultado por completo. Al revisarlo encontramos que 20 de las 59 negaciones que VADER reconoce son palabras vacías según nltk, de modo que nuestra limpieza las elimina. Los tweets del conjunto que contienen alguna negación son 857, el 11.26% del total.

El efecto es que la polaridad se invierte. La frase *never seen anything this bad* pasa de un compound de -0.628 sobre el texto crudo a +0.431 sobre el texto limpio, simplemente porque *never* desapareció. Por eso calculamos el sentimiento sobre el texto crudo, al que solo le quitamos URLs y artefactos de codificación, conservando deliberadamente mayúsculas, puntuación, negaciones y emoticones. Es un pipeline distinto al de la sección 3 porque persigue un objetivo distinto: aquel buscaba reducir el vocabulario para el clasificador, este busca preservar la carga emocional.

8.2 La pregunta sobre los emoticones

El enunciado pregunta si conviene dejar los emoticones y analizarlos. Lo medimos sobre el conjunto en lugar de responder de memoria, y el hallazgo nos resultó inesperado. El conjunto de datos no contiene un solo emoji Unicode. Los tweets con emoticones ASCII, del tipo dos puntos seguidos de paréntesis, son 95, el 1.25% del corpus. En cambio, el 9.16% de los tweets contiene caracteres fuera del rango ASCII, y al inspeccionarlos resultaron ser mojibake: secuencias producidas por una conversión defectuosa entre CP1252 y UTF-8 en algún punto de la recolección.

Eso reencuadra la pregunta. El paso de eliminación de caracteres no ASCII de la sección 3 en realidad nunca quitó emojis en este corpus, porque no los hay; lo que quitó fue basura de codificación, y hace bien en quitarla. Respondiendo a lo que pide el enunciado, sí vale la pena conservar los emoticones ASCII, porque VADER los puntúa y modifican el resultado sin costo alguno, pero su efecto agregado es marginal al aparecer en tan pocos tweets. El hallazgo relevante de esta sección es otro: lo que de verdad había que rescatar del preprocesamiento no eran los emoticones sino las negaciones. Los emoticones eran la pregunta visible y las negaciones el problema real.

8.3 Clasificación de palabras y puntuación de cada tweet

El léxico de VADER aporta 4,169 términos negativos y 3,333 positivos, en total 7,502 términos con carga emocional. La categoría neutra sale vacía al clasificar el léxico en sí mismo,

porque el léxico solo almacena términos con carga y todo lo que no aparece en él se considera neutro por omisión. Más informativo que el léxico completo resultó observar qué palabras con carga emocional usa realmente nuestro corpus, como muestra la figura 11.

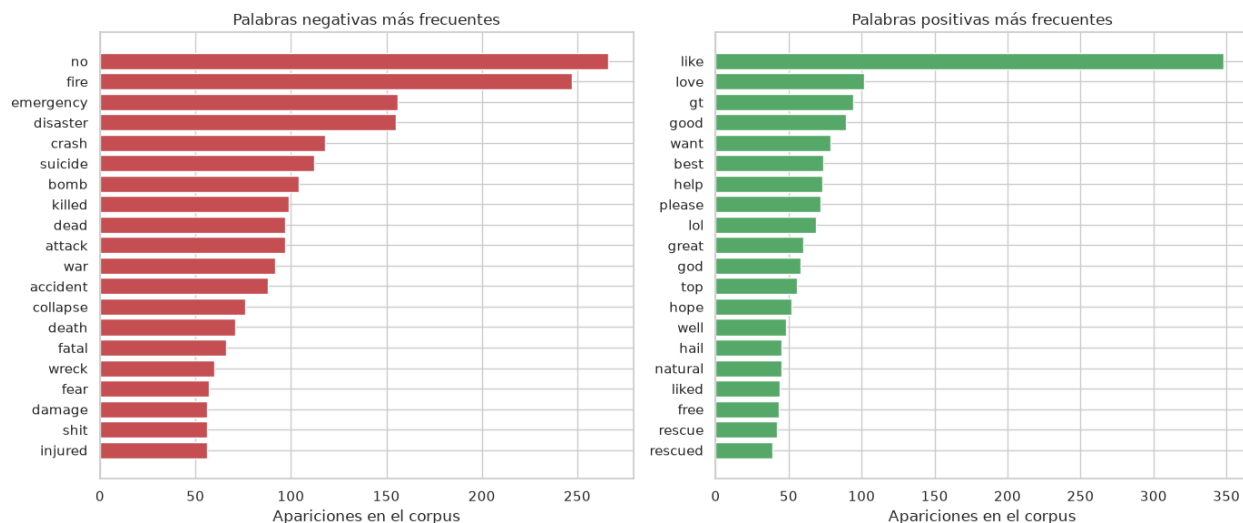


Figura 11. Palabras negativas y positivas más frecuentes en el corpus, con su valencia.

El vocabulario negativo del corpus está dominado por términos de catástrofe como fire, disaster, emergency, killed y crash, coherente con que casi la mitad del conjunto describe desastres. El positivo es más genérico y conversacional, con like, love, good y best. Vale notar que fire aparece como negativa con valencia -1.4, que es la misma ambigüedad arrastrada desde el análisis de frecuencias: en la clase de no desastre fire se usa como elogio y el léxico no lo distingue.

Para puntuar cada tweet implementamos el conteo directo que pide el enunciado, calculando la diferencia entre el número de palabras positivas y negativas dividida entre su suma, lo que deja el resultado en el intervalo de -1 a 1, y etiquetando como positivo, negativo o neutral según caiga fuera o dentro de una banda de 0.05 alrededor de cero. En paralelo conservamos el compound de VADER, porque corrige justamente los casos donde el conteo se equivoca. La frase this is not good at all contiene una sola palabra del léxico, good, así que el conteo la declara positiva, mientras que VADER lee la negación previa y la declara negativa, que es lo correcto.

Los dos métodos coinciden en el 83.4% de los tweets. Donde más difieren es en la banda neutral, porque el conteo declara neutrales bastantes tweets que VADER sí carga, al ignorar los intensificadores y el énfasis de la puntuación y las mayúsculas. En conjunto, el corpus resulta predominantemente negativo: 48.9% de los tweets según VADER, frente a 24.8% positivos y 26.3% neutrales.

9. Sentimiento por categoría de tweet

9.1 Los diez tweets más negativos y los diez más positivos

De los diez tweets más negativos del corpus, ocho pertenecen a la categoría de desastre real. Son atentados con explosivos, tiroteos y ataques con víctimas mortales, redactados en registro noticioso y con términos como suicide bomber, killed y dead acumulados en un mismo tweet.

Los dos que no son desastre merecen comentario porque exponen los límites del método. El más negativo del corpus entero, con un compound de -0.9883, es un tweet que repite la palabra wreck trece veces seguidas. No describe nada: la puntuación se satura por acumulación de un mismo término negativo, de modo que es un artefacto del método y no un tweet especialmente sombrío. El otro narra el hallazgo de un pez muerto en un lago en tono conversacional; contiene dead y poor, suficiente para hundir la puntuación, pero es una anécdota casual.

Con los diez más positivos el reparto se invierte: ocho de los diez son de la categoría de no desastre, y corresponden a promociones, conversación entre usuarios y entusiasmo por música o televisión. Los dos etiquetados como desastre real son los más interesantes, porque cuesta justificar la etiqueta. Uno usa storm como metáfora de consuelo en una frase de aliento y el otro emplea derailment en sentido figurado dentro de una charla amistosa. Ninguno informa de un desastre. La diferencia con los casos de la sección 7 es que aquí el error no es del modelo sino de la etiqueta original, lo que da una idea del techo de exactitud alcanzable sobre estos datos.

9.2 ¿Son más negativos los tweets de desastre real?

La respuesta es afirmativa y la diferencia es sólida. La tabla 6 resume los estadísticos por categoría y la figura 12 muestra las distribuciones.

Categoría	N	Media	Mediana	Desv.
Desastre real	3,271	-0.2667	-0.3182	0.4289
No desastre	4,342	-0.0611	0.0000	0.4582

Tabla 6. Estadísticos del compound de VADER por categoría de tweet.

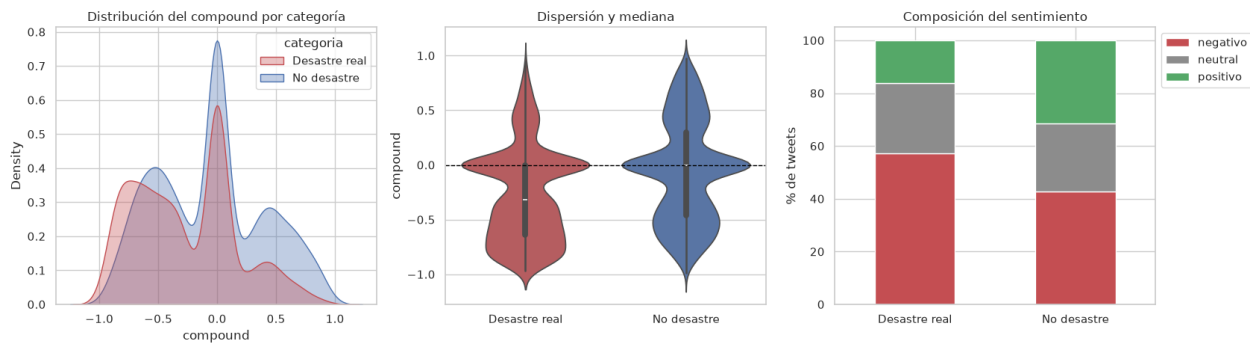


Figura 12. Distribución del sentimiento en cada categoría de tweet.

La mediana marca el contraste todavía mejor que la media: -0.318 en la categoría de desastre frente a 0.000 en la otra, es decir que el tweet típico de no desastre es neutro mientras que el típico de desastre ya carga negatividad. El 57.1% de los tweets de desastre son negativos, contra el 42.7% de los otros, y la proporción de positivos casi se duplica en sentido contrario, 31.4% frente a 16.1%.

Como la distribución del compound es bimodal y está lejos de ser normal, usamos como prueba principal la U de Mann-Whitney, que compara rangos y no supone normalidad. Obtuvimos un valor p de aproximadamente $1e-80$, así que la diferencia no es casualidad de muestreo. Añadimos la t de Welch como referencia, con un valor p de $1.7e-87$, y dos medidas de tamaño de efecto, porque con más de 7,000 observaciones casi cualquier diferencia sale significativa y el valor p por sí solo no indica si la diferencia importa. La d de Cohen resultó -0.4611, lo que la sitúa como efecto mediano.

Ese matiz nos pareció el dato más importante de la sección. Que el 42.7% de los tweets de no desastre también sean negativos significa que la negatividad no distingue un terremoto de una queja sobre el tráfico o de una mala racha deportiva. Es una señal complementaria a la del vocabulario y no un sustituto, por lo que esperábamos del siguiente ejercicio una mejora moderada del clasificador antes que un salto.

9.3 Una observación sobre la calidad de los datos

Al revisar los tweets extremos aparecieron textos repetidos y decidimos contarlos. Encontramos 179 tweets con texto duplicado, el 2.35% del corpus, distribuidos en 69 textos distintos. En 18 casos el mismo texto exacto está etiquetado a la vez como desastre y como no desastre, de modo que ningún modelo puede acertar en ambas copias. Es una cantidad pequeña frente a 7,613 tweets, pero ayuda a interpretar el desempeño del clasificador: parte del error que reportamos no proviene del modelo sino de etiquetas inconsistentes y de casos genuinamente ambiguos como los dos desastres positivos comentados arriba.

10. La variable de negatividad y el reentrenamiento

Definimos la negatividad de cada tweet como el compound de VADER con el signo invertido, de modo que la variable queda acotada entre -1 y 1 y se lee en la dirección que su nombre indica, con 1 como máxima negatividad. La calculamos sobre el texto crudo por lo establecido en la sección 8. Conviene señalar que se obtiene tweet por tweet contra un léxico fijo, sin ajustar nada a partir del conjunto de datos, de modo que no existe riesgo de fuga de información entre entrenamiento y prueba.

10.1 Montaje del experimento

El modelo de la sección 6 recibía una serie de texto. Para incorporar una columna numérica hubo que reexpresarlo sobre un DataFrame mediante un ColumnTransformer que aplica TF-IDF a la columna de texto y estandarización a las numéricas. La estandarización es necesaria porque los valores TF-IDF están acotados entre 0 y 1, y sin escalar, una variable en otro rango recibiría una penalización de regularización desproporcionada.

Antes de comparar nada verificamos que el cambio de montaje no hubiera alterado el modelo. El nuevo esquema alimentado solo con texto reproduce las métricas de la sección 6 hasta el sexto decimal, con exactitud 0.804990 y F1 0.769231. Sin esa comprobación habríamos corrido el riesgo de medir el andamiaje en lugar de la variable.

10.2 Resultados del reentrenamiento

Comparamos tres modelos idénticos en algoritmo, hiperparámetros y semilla, que difieren únicamente en las columnas que reciben, sobre veinte particiones aleatorias y de forma pareada. La tabla 7 recoge los resultados.

Modelo	F1 medio	Desv.	Gana	Valor p
Solo texto	0.7622	0.0091	—	—
Más negatividad	0.7597	0.0091	8/20	0.075
Más negatividad y conteos	0.7606	0.0085	6/20	0.232

Tabla 7. Efecto de incorporar la negatividad sobre veinte particiones aleatorias.

La respuesta a la pregunta del enunciado es que la inclusión de la variable no mejoró el modelo. El F1 pasa de 0.7622 a 0.7597, una diferencia de -0.0026 que no alcanza significancia estadística y que gana en apenas 8 de 20 particiones. La lectura honesta no es que la variable empeore el modelo, sino que la diferencia es indistinguible de cero con una tendencia levemente negativa. Añadir además los conteos de palabras positivas y negativas tampoco ayuda.

El resultado contradice aparentemente lo encontrado en la sección 9, donde la negatividad separaba las clases con un valor p del orden de $1e-80$, así que dedicamos esfuerzo a entender por qué.

10.3 Por qué una variable con señal no aporta nada

Consideramos dos explicaciones posibles, que llevan a conclusiones distintas: que la variable no tuviera señal, o que la tuviera pero el modelo ya la poseyera por otra vía. La primera quedó descartada rápidamente. Entrenada por sí sola, sin texto alguno, la negatividad alcanza un ROC-AUC de 0.629, claramente por encima del 0.5 que correspondería al azar. Y en el modelo combinado recibe un coeficiente de 0.3936, que supera en magnitud al 95.9% de los 52,815 coeficientes asociados a términos de texto. No es una variable que el modelo esté ignorando.

La segunda explicación resultó ser la correcta. Al revisar los treinta términos que más empujan hacia la categoría de desastre, encontramos que siete de ellos tienen carga de sentimiento en el léxico de VADER: fire, killed, suicide, accident, disaster, attack y severe. Las palabras que hacen negativo a un tweet son en buena medida las mismas que lo hacen predecible como desastre. La negatividad es entonces un resumen escalar de información que el modelo ya tiene desagregada término por término.

Esa explicación admite una predicción falsable, y nos pareció que valía la pena someterla a prueba: si el problema es redundancia, la negatividad debería ayudar cuando el modelo dispone de poco vocabulario y dejar de hacerlo conforme el vocabulario crece hasta contener esa misma información. Repetimos el entrenamiento limitando el número de términos del vectorizador a distintos tamaños, promediando ocho particiones en cada caso. La figura 13 muestra el resultado.

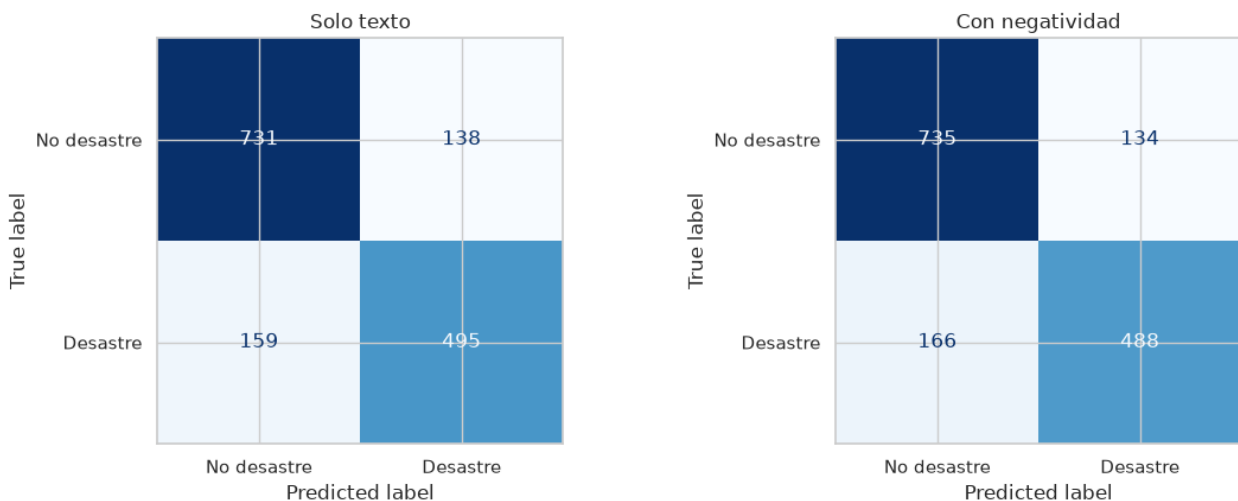


Figura 13. Aporte de la negatividad según el tamaño del vocabulario del modelo.

La predicción se cumplió con nitidez. Con 50 términos la negatividad aporta 0.059 de F1, que es una mejora sustancial. El aporte cae a 0.022 con 100 términos, a 0.008 con 250, a 0.0006 con 1,000 y se vuelve negativo con el vocabulario completo. Esto confirma que la variable contiene información real y útil, pero es información que el TF-IDF ya captura por sí solo en cuanto dispone de suficientes términos. Con vocabulario completo, añadirla solo agrega un parámetro más que estimar y el modelo paga ese costo sin recibir nada a cambio.

En consecuencia, el modelo que consideramos definitivo es el de solo texto de la sección 6. Es igual de bueno, más simple y no arrastra la dependencia de nltk y del léxico de VADER en el momento de la predicción. La variable de negatividad conserva todo su valor descriptivo, como mostró la sección 9, pero no aporta valor predictivo sobre este modelo.

11. Conclusiones

El mejor modelo que obtuvimos es una regresión logística sobre TF-IDF de unigramas y bigramas, con exactitud de 0.805, F1 de 0.769 sobre la clase de desastre y ROC-AUC de 0.869. Lo seleccionamos tras evaluar doce configuraciones, correspondientes a cuatro algoritmos por tres representaciones de n-gramas, y una búsqueda en malla de 120 combinaciones sobre el algoritmo ganador.

El hallazgo metodológico que consideramos más valioso es que una sola partición de los datos no basta para elegir entre modelos parecidos. En nuestro caso, la partición inicial señalaba como ganador a un modelo que perdía sistemáticamente al repetir la comparación sobre veinte particiones. Adoptar la comparación pareada como criterio nos evitó ese error y volvió a resultar decisiva al evaluar la variable de negatividad.

Sobre el tratamiento del contexto, la conclusión fue contraria a lo que anticipábamos. Los bigramas producen agrupaciones legibles para un lector humano, pero no mejoran la predicción, porque los tweets son demasiado cortos y la señal que separa las clases es léxica antes que sintáctica. Los trigramas resultan directamente perjudiciales al ajustarse a titulares replicados por cuentas automatizadas.

El análisis de sentimiento dejó dos aprendizajes. El primero es que el preprocesamiento debe diseñarse en función del objetivo: la limpieza pensada para el clasificador destruye la señal que necesita el analizador de sentimiento, porque elimina 20 de las 59 negaciones que VADER reconoce e invierte la polaridad en el 11.26% de los tweets. El segundo es que los tweets de desastre real son efectivamente más negativos, con un tamaño de efecto mediano, pero que esa negatividad no basta para separarlos porque el 42.7% de los tweets de la otra categoría también son negativos.

Finalmente, incorporar la negatividad como variable no mejoró la clasificación, y la investigación de por qué nos resultó más instructiva que el resultado en sí. La variable tiene

señal genuina, pero es redundante con el vocabulario que el modelo ya utiliza. El barrido sobre el tamaño del vocabulario lo demuestra de forma directa y constituye, a nuestro juicio, el experimento más informativo de todo el laboratorio.

Como limitaciones, el modelo sigue siendo vulnerable al lenguaje figurado, especialmente en tweets cortos donde un solo término con coeficiente alto decide la predicción sin contrapeso. Además, el conjunto de datos contiene 179 tweets duplicados, 18 de ellos con etiquetas contradictorias, lo que fija un piso de error que ningún clasificador puede cruzar. Una línea de trabajo natural sería sustituir la bolsa de palabras por representaciones contextuales, capaces de distinguir el sentido literal del figurado a partir del entorno de cada término.

12. Referencias

Hutto, C. J., y Gilbert, E. (2014). VADER: A Parsimonious Rule-based Model for Sentiment Analysis of Social Media Text. Proceedings of the Eighth International AAAI Conference on Weblogs and Social Media. Utilizado a través del módulo `nlk.sentiment.vader` para todo el análisis de sentimiento de las secciones 8, 9 y 10.

Bird, S., Klein, E., y Loper, E. (2009). Natural Language Processing with Python. O'Reilly Media. Biblioteca `nlk`, empleada para la lista de palabras vacías en inglés del pipeline de limpieza y para el léxico de sentimiento.

Pedregosa, F. y otros (2011). Scikit-learn: Machine Learning in Python. Journal of Machine Learning Research, 12, 2825-2830. Utilizada para la vectorización TF-IDF, los cuatro algoritmos de clasificación, la búsqueda en malla y las métricas de evaluación.

Jurafsky, D., y Martin, J. H. (2014). N-Grams. Speech and Language Processing. Consultado como referencia conceptual para el análisis de n-gramas de la sección 4.

Kaggle (2019). Natural Language Processing with Disaster Tweets. Conjunto de datos de la competencia, disponible en kaggle.com/competitions/nlp-getting-started. El archivo `train.csv` utilizado tiene suma de verificación SHA256 61111c6dc31eaffa34d1e1fa62e2395325c9bc3b38bba1941a5f1ed9b3fa60df.

Mueller, A. (2020). WordCloud for Python. Empleada para las nubes de palabras de la figura 6. Las bibliotecas `pandas`, `numpy`, `scipy`, `matplotlib` y `seaborn` se utilizaron para la manipulación de datos, las pruebas estadísticas y las visualizaciones de todo el informe.